SIMATS ENGINEERING
ASSESSMENT TOOL 1

22

COURSE CODE: CSA04
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

***CO1**: Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Christina.y *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Christina.y

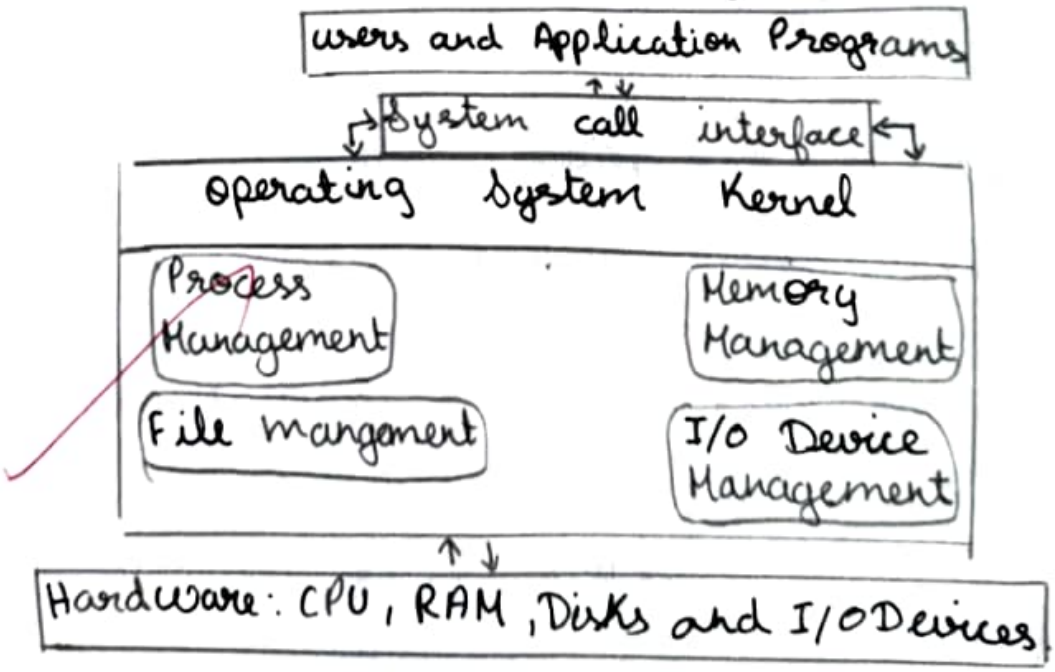
Name : Christina
Regno : 1965260124
course code : CSA0408
Subject : operating System

22/25 by

1. a) **Multi - user OS** : Allows multiple users to access the same computer system simultaneously
2. **Multitasking OS** : Executes multiple tasks by sharing CPU time among processes.
3. **Multiprocessing OS** : Uses two or more CPU cores or processors to execute processes.
4. **time - sharing OS** : Allocates a small time slice to each process or user to provide interactive access.

A combination of multi-user, multitasking, time-sharing OS is most suitable because it supports multiple users, runs several processes, and shares resources efficiently.

b)



c) In a UNIX ALINOX operating system the sequence is

1. fork() - creates a new child process
2. exec() - loads and executes a new program in the a child process
3. wait() - Allows the parent to wait for the child to finish
4. exit() - Terminates the process

Eg sequence : fork() → exec() → exit()

---

2. File and Device Management

a) categorized system calls used for file and device management

File Managment System calls

- create() - Creates a file
- open() - opens a file
- read() - Reads file data
- write() - Writes data to a file
- close() - closes a file
- unlink() - Removes a file name.

Device Management System calls

- openI+ opens a device
- read() - Reads input from a device
- write() - sends output to o device
- ioch() - Performs device - specific control operations.
- close() - closes the device.

In Contents Expl...

User Application

↓

System call Interface (open, read, write, close)

↓

Virtual File System (VFS)

↓

File System / Device Driver

↓

storage Device / I/O Hardware

c)
```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main() {
    int fd;
    char buffer[100];
    fd = open("data.txt", O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    int n = read(fd, buffer, sizeof(buffer));
    if (n == -1) {
        perror("read");
        close(fd);
        return 1;
    }
    write(STDOUT_FILENO, buffer, n);
    close(fd);
    return 0;
}
```

3.
a) A Real-time operating System (RTOS) is suitable for controlling industrial machinery

Features of RTOS:

1. Deterministic response: Responds to events within predictable time limits
2. Priority-based scheduling: gives priority to critical
3. low latency: & minimizes delays in processing events.
4. Reliability: Provides stable operation for critical time applications
5. task synchronization

b)



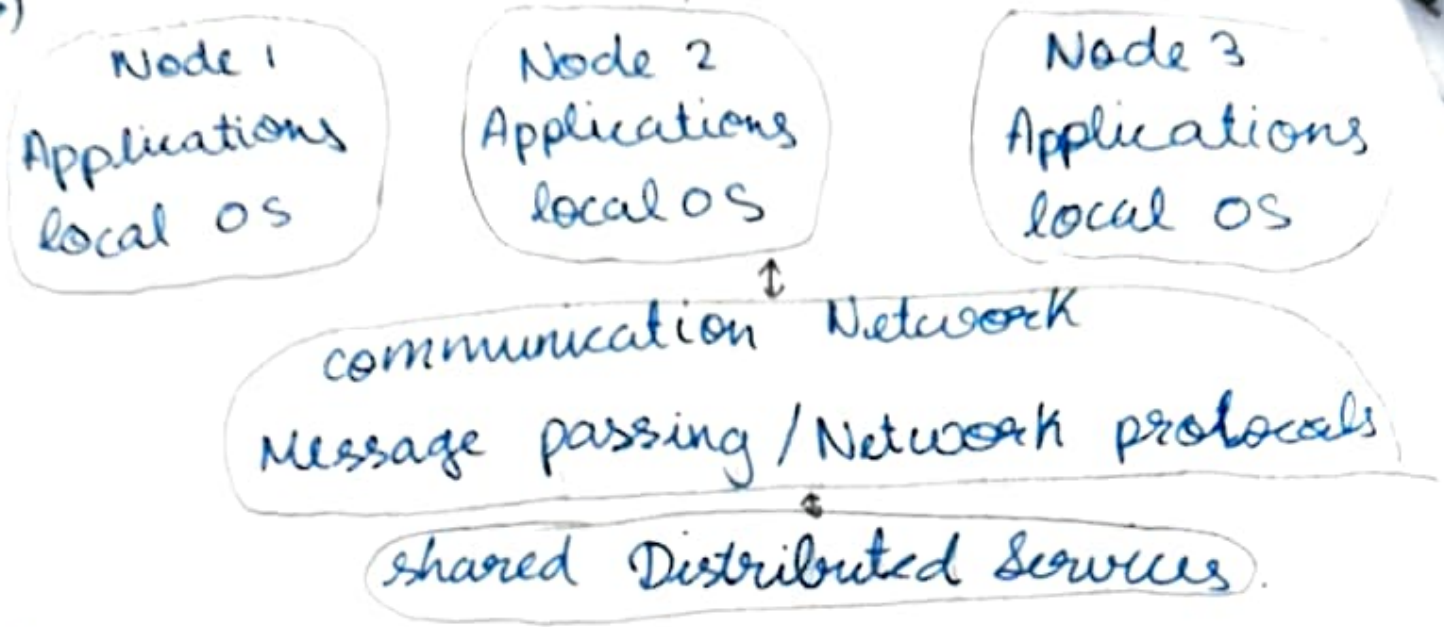
System calls for process synchronization

1. Sem - wait() - waits for a semaphore and may block a process until a resource is available.
2. Sem - post() - Increases a semaphore value and may wake a waiting process
3. pthread - mutex - lock() - locks a mutex to protect a shared resource.
4. pthread - mutex - unlock() - unlocks the mutex after using the resource.
5. pthread - cond - wait() - waits for a condition to become true.

4. The suitable type is a Distributed operating system (DOS).

Distributed OS	centralized OS
1. Uses multiple interconnected computers.	Primarily manages one central computer
2. Resources are distributed across nodes	Resources are concentrated in the central system
3. Better scalability	May require upgrading the central system
4. Communication occurs over a network	Resource mangement is mainly local to the central system

b)

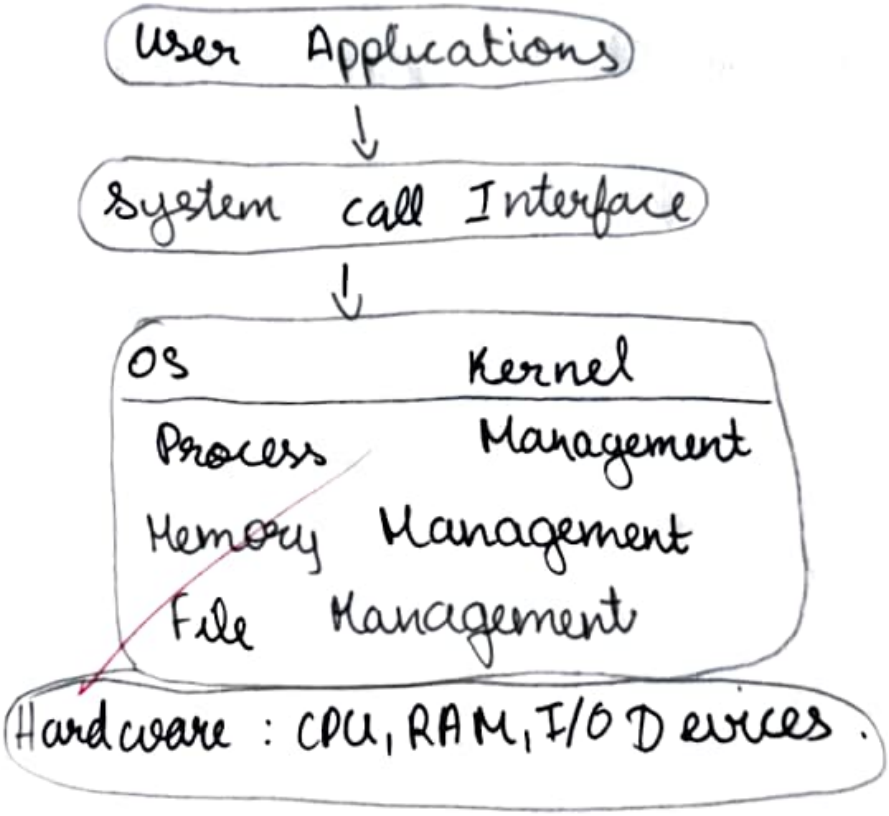


c) One common IPC method is message passing - sequence:

1. pipe() - creates a communication channel
2. fork() - creates a child process that enherits the pipe descriptors.
3. write() - the sender writes data into the pipe.
4. read() - Receiver reads data from the pipe.
5. close() - closes unused pipe descriptors.

Major functions of an OS

1. **Process Management**: Creates, schedules and and terminates processes
2. **Memory Management**: Allocates and frees memory
3. **Device Management**: Controls I/O devices using drivers.
4. **File Management**: Creates reads and writes files.
5. **Security**: Protects data and system resources.



c) System calls for memory and device management

Memory management

- Mmap() - maps memory
- munmap() - Releases mapped memory

Device management

- open() - opens a device
- read() - Reads device data
- write() - Writes data to a device
- close() - closes a device.